

Comprensión de listas o Comprehension Lists

Colecciones

Una colección es una estructura que como su nombre indica nos permite almacenar varios elementos. Existen diferentes tipos de colecciones, cada una con diferentes características lo que lo hace más indicado para según qué tipo de almacenamiento.

El tipo más sencillo son las **Tuplas** (tuple), un conjunto invariable de elementos, no tienen que ser del mismo tipo.

Veremos también las **Listas** (list) nos permiten guardar también múltiples objetos y de cualquier tipo, con la peculiaridad de que están ordenados. Disponen de todos los mecanismos para trabajar dinámicamente con ellas, añadiendo o quitando elementos, reordenando.

Otro tipo muy importante son los **Diccionarios** (dict), donde tenemos un conjunto de pares que no están ordenados, formados por una clave (key) y un valor (value). La clave tiene que ser única.

También existen los **Conjuntos** (sets), que almacenan valores sin ordenar y que no pueden estar repetidos. Sólo se pueden guardar elementos inmutables (que no cambian) y que de los que se puede obtener un valor **hash** (algo similar a un id único)

Para cada caso escogeremos la estructura que nos facilite más la programación, donde la propia naturaleza de la estructura nos será muy útil.

Por ejemplo para guardar una lista de propiedades, parece más interesante utilizar un diccionario, donde guardaremos de forma natural el nombre de la propiedad y su valor.

Si necesitamos mantener varios elementos sin que se produzca repetición, parece que un conjunto sería lo más indicado.

Tuplas

El tipo más sencillo de colección son las **Tuplas**, que son un conjunto invariable de elementos, es decir, una vez creadas no podemos modificarlas, ni añadir ni quitar elementos. Los elementos no tienen que ser del mismo tipo, ni tampoco podemos cambiarlos de valor, es decir sus elementos también tienen que ser invariantes, por lo que no pueden ser variables.

¿Qué datos podemos usar que no necesiten cambio? Por ejemplo los días de la semana:

```
>>> dias = ('L', 'M', 'X', 'J', 'V', 'S', 'D')
```

No hay ningún problema en que se repitan elementos dentro de la tupla.

Las definimos poniendo los elementos que la forman entre paréntesis:

```
>>> coordenadas = (1,3,4)
>>> elementos = ('a',7,'hola qué tal')
```

Podemos acceder a un elemento de la tupla con su índice que irá entre 0 y N-1 siendo N el número de elementos de la tupla, indicando la posición del elemento entre corchetes:

```
>>> elementos[0]
'a'
```

Si intentamos acceder a un elemento más allá del último obtendremos una excepción del tipo *IndexError*.

También podemos acceder a los elementos con índices negativos, en cuyo caso se entiende que índice -1 corresponde al último y accederemos hacia atrás:

```
>>> elementos[-1]
'hola qué tal'
>>> elementos[-2]
7
```

Podemos *sumar* tuplas usando el operador "+" y obtendremos una nueva con los elementos de las 2 sumadas.

Si usamos el operador multiplicación "*" lo que haremos será repetirla N veces.

También podemos acceder a una parte de la tupla usando el índice de elemento inicial y del final separados por ":", obtendremos una lista con los elementos seleccionados:

```
>>> elementos[1:2]
(7, 'hola qué tal')
```

También podemos omitir el número inicial o el final si queremos que empiece en el

principio o acabe al final de la tupla:

```
>>> festivos = dias[5:]
>>> laborables = dias[:5]
```

Se suele llamar rodaja (slice) a esta parte que tomamos de una colección mayor.

También podemos usar un tercer entero para indicar cada cuantos elementos queremos iterar.

¿Te recuerda a algo? Sí: la función **range** es un generador de tuplas de números.

Iterando

Podemos recorrer los elementos de una tupla usando el bucle *for*:

```
valores = (1,5,2,7) # Crea una tupla llamada valores con los 4 números
indicados
for valor in valores: # itera entre los 4 números de la tupla "valores"
recuperando cada vez uno en la variable "valor"
    print(valor) # imprime el valor que se recuperado en "valor" y vuelve a
repetir el bucle hasta que no queden más valores
```

Podemos conocer el tamaño de una tupla usando el método *len*, lo que nos permite recorrer la tupla usando un bucle *while*

```
valores = (1,5,2,7)
contador = 0 # Usaremos "contador" como un índice para ir recuperando el
elemento de la posición "contador" desde la posición 0 (el primero)
while contador < len(valores): # hasta la posición indicada por el tamaño
"len(valores)"
    print(valores[contador]) # Accedemos al elemento de la posición
"contador" y lo imprimimos
    contador += 1 # pasamos índice siguiente y repetimos
```

Podemos saber si un elemento está en una tupla usando la palabra reservada *in*:

```
>>> 4 in valores
False
```

Que nos dará True o False según si el elemento está o no en la tupla.

```
>>> 'X' in festivos
False
```

Podemos usar esta funcionalidad cuando queramos hacer que el usuario seleccione una opción entre varias y e caso de que elija una opción distinta le volveremos a preguntar:

```
print('1 - Opción primera')
print('2 - Opción segunda')
print('3 - Opción tercera')
print('4 - Opción cuarta')
```

```

print('0 - Para salir')
opcion = input('Seleccione entre estas opciones')
while opcion not in ('0','1', '2', '3', '4'):
    print('Opción no válida')
    opcion = input('Seleccione entre estas opciones')
print(f'Seleccionada la opción {opcion}')

```

Código comprobación de opciones

También podemos saber cuántas veces aparece un elemento con el método *count()*:

```

>>> valores.count(5)
1

```

Python usa internamente las tuplas en muchos casos, por eso podemos utilizarlas también cuando queremos trabajar con varias variables al mismo tiempo:

```

>>> a,b = 1,2 # Para asignar valores a varias al mismo tiempo
>>> a,b = b,a # Para intercambiar sus valores
>>> a
2
>>> b
1

```

Una vez que sabemos que un elemento está en la tupla podemos encontrar su posición utilizando el método **index**.

```

>>> ganadores = ('Brasil','Italia','Brasil')
>>> ganadores.count('Brasil')
2
>>> ganadores.index('Brasil')
0

```

Si algún elemento está más de una vez podemos hacer que la búsqueda se realice a partir de un elemento indicando el índice a partir del que buscar

```

>>> ganadores.index('Brasil',1)
2

```

Ejemplo de uso de tuplas: BuzzWord

NASA - MSC
Houston, Texas

MSC MEMORANDUM

10 October 1966

SUBJECT: Buzzword Generator

1. The Buzzword Generator consists of three columns of buzzwords numbered zero to nine.

Column 1	Column 2	Column 3
0. integrated	0. Management	0. Options
1. total	1. organizational	1. Flexibility
2. systematized	2. monitored	2. capability
3. parallel	3. reciprocal	3. mobility
4. functional	4. digital	4. programming
5. responsive	5. logistical	5. concept
6. optimal	6. transitional	6. time-phase
7. synchronized	7. incremental	7. projection
8. compatible	8. third-generation	8. hardware
9. balanced	9. policy	9. contingency

2. The procedure is simple. Think of any three digit number at random. Then select the corresponding buzzword from each column. Put them together and WHAM! POW! ZAP! You sound just like you know what you're talking about.
3. Take for instance the number 257. Take word two from column one, word five from column two and word seven from column three. You now have "systematized logistical projection". You don't know what it means but don't worry, neither do "they".
4. Would you prefer "balanced incremental flexibility". Possibly "parallel reciprocal options". Or maybe "integrated transitional contingency". How about "functional third-generation hardware" and "optimal management mobility". Now that ought to do the trick.
5. The important thing is that the Buzzword Generator provides the user with the perfect aid for preparing anything on the subject of spaceflight. Automatically you have one thousand different combinations, all of which will give you that proper ring of decisive, progressive, knowledgeable, authority.

Vamos a hacer ahora un programa que ya usaban en la mismísima NASA allá por 1966. Se trata de crear un generador de palabros técnicos (BuzzWord), con el fin de poder rellenar informes de jerga.

A partir de 3 columnas de 10 términos generaremos un palabro técnico escogiendo aleatoriamente 1 palabra de cada columna.

Vamos a utilizar tuplas para guardar la colección de elementos que no cambian.

También vamos a usar el generador de números aleatorios para elegir una palabra de cada columna usando la función `randint(0, 9)` que nos va a dar un número aleatorio entre 0 y 9, con los extremos incluidos. Para ello tenemos que incluir el módulo *random*, que es lo que hacemos en la línea `"from random import randint"`.

Más adelante veremos con más detalle el uso de módulos, ahora mismo es suficiente con saber que un módulo es un conjunto de código externo que podemos usar desde nuestro programa.

Código del Generador de Palabros

Ejemplo de matrices con tuplas: Cálculo del IRPF

Vamos a hacer ahora otro ejemplo que también usa tuplas: un programa que calcula nuestras retenciones IRPF. A partir del dato de nuestro sueldo calcularemos siguiendo la tabla de retenciones, que almacenaremos en nuestro programa en una matriz formada por tuplas de tuplas de valores:

```
>>> tupla1 = (1,2)
>>> tupla2 = (3,4)
>>> matriz_tupla = (tupla1,tupla2)
```

Accederemos a un elemento usando 2 índices:

```
>>> matriz_tupla[1][0]
```

En nuestro ejemplo usaremos esta matriz de tuplas para guardar los distintos tramos del IRPF:

```
tramos = ((12450, 0.19),
          (20200, 0.24),
          (35200, 0.30),
          (60000, 0.37),
          (300000, 0.45),
          (10e20, 0.47))
```

Código de Cálculo de IRPF

Listas

Una lista es una colección en la que los elementos que pueden ser de distintos tipos, están ordenados.

Características de las listas:

- No todos los elementos tienen que ser del mismo tipo.
- La lista es dinámica y puede cambiar de tamaño, es decir, podemos añadir y quitar elementos.
- Los elementos están ordenados, en cuanto a su posición en la lista. No es que por defecto se les aplique un orden al insertarlos. Además dado que pueden ser de distintos tipos, no siempre se puede establecer un orden según su valor.

Si has programado en otro lenguaje, las listas te pueden recordar a los arrays, pero en Python son mucho más dinámicas y versátiles.

Para declararla lo haremos de la siguiente forma:

```
mi_lista = [ 10, "hola", "Pepe",25]
```

Accederemos a cada elemento usando su posición, empezando en 0 y terminando en el elemento N-1, siendo N el número de elementos de la lista.

```
print(mi_lista[2])
```

Podemos modificar el elemento que está en una posición dada:

```
mi_lista = [ 10, "hola", "Pepe",25]
```

```
print("inicialmente",mi_lista[2])
```

```
mi_lista[2] = "Juan"
```

```
print("ahora es",mi_lista[2])
```

También podemos usar un índice negativo siendo **-1** el elemento de la última posición y así hasta llegar al primero.

Si intentamos acceder a un elemento más allá de la última o de la primera obtendremos una excepción **IndexError**

Podemos crear una lista vacía con `lista = []`

SubListas

También podemos recuperar varios elementos de una lista, es lo que se llama una **sublista**. Para ello indicaremos un rango de valores, que son 2 valores numéricos separados por ":", de la siguiente forma **principio:final**, siendo **principio** el índice del primer elemento de la sublista y **final** el índice posterior al último que vamos a recuperar.

```
>>> print(mi_lista[0:2])  
[10, 'hola']
```

Si omitimos el primer valor se entenderá que nos referimos al primer valor de la lista

```
>>> print(mi_lista[:2])  
[10, 'hola']
```

y si omitimos el segundo que queremos llegar hasta el elemento final

```
>>> print(mi_lista[2:])  
['Juan', 25]
```

Si omitimos los dos tendremos la lista entera

```
>>> print(mi_lista[:])  
[10, 'hola', 'Juan', 25]
```

También podemos incluir un tercer número en la sublista (separado por otro :) que indicará cada cuantos elementos contamos al elegirlos (lo que sería el "paso"). Por defecto este número es 1, pero podría ser otro.

```
>>> print(mi_lista[::2])  
[10, 'Juan']
```

Una utilidad de este tercer parámetro es invertir el orden de nuestra lista usando **-fi**. Por ejemplo:

```
>>> print(mi_lista[::-1])  
[25, 'Juan', 'hola', 10]
```

Operaciones sobre listas

Si nuestra lista está formada por elementos homogéneos (todo números o todo texto) podremos realizar una serie de operaciones sobre ellas:

- **Ordenación:** podemos ordenar una lista con *mi_lista.sort()*. Una vez ordenada sabremos que el primero o el último serán el mínimo o el máximo valor.
 - Podemos ordenarla en orden inverso con *mi_lista.sort(reverse=True)* para . Conviene recalcar que la operación se realiza sobre la propia lista, no es que la llamada nos devuelva una lista ordenada.
 - Por defecto la ordenación de cadenas distingue entre mayúsculas y minúsculas,

```
>>> palabras = ['hola', 'ola', 'Hola', 'hOLA']  
>>> palabras.sort()  
>>> palabras  
['Hola', 'hOLA', 'hola', 'ola']
```

- Podemos hacer que no dependa de mayúsculas/minúsculas usando una función de ordenación diferente, lo que indicaremos con el argumento **key**, por ejemplo usando la función *str.lower* (que usará todos los caracteres como minúsculos antes de hacer la comparación)

```
>>> palabras = ['hola', 'ola', 'Hola', 'hOLA']
>>> palabras.sort(key=str.lower)
>>> palabras
['hola', 'Hola', 'hOLA', 'ola']
```

- Para que no dependa de si la primera letra es mayúscula usaremos *str.capitalize* que usa el primer carácter de cada cadena como mayúscula

```
>>> palabras = ['Hola', 'ola', 'hola', 'hOLA']
>>> palabras.sort(key=str.capitalize)
>>> palabras
['Hola', 'hola', 'hOLA', 'ola']
```

- Obtener el máximo aplicando la función **max** *max(mi_lista)*.
- Obtener el mínimo aplicando la función **min** *min(mi_lista)*.
- Calcular su longitud con **len** *len(mi_lista)*.

Búsquedas

Una funcionalidad que usaremos muchos son las búsquedas, que podremos hacer con la palabra reservada **in**. Obtendremos como resultado un booleano, siendo *True* que se ha encontrado y *False* en caso contrario.

Conviene destacar que se distingue entre mayúsculas y minúsculas.

```
>>> mi_lista = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
>>> 'Juan' in mi_lista
True
```

Una vez que sabemos que el elemento está en la lista podemos preguntar en qué posición con **index**:

```
>>> posicion = mi_lista.index("Juan")
0
```

Si intentamos buscar un elemento que no está se produce una excepción de tipo **ValueError**.

También podemos contar el número de veces que se repite un valor con **count**:

```
>>> mi_lista.count("Juan")
1
>>> mi_lista.count("juan")
0
```

Como vemos podemos usar la función *count* para ver si un elemento está o no en una lista.

Edición en listas

Hemos visto cómo cambiar, buscar, contar o acceder a elementos de una lista. Pero se pueden hacer más operaciones como las siguientes, a las que llamaremos a partir de nuestra lista original

- **append(elemento)**: Añade un elemento al final de la lista.
- **clear()**: borra todos los elementos de una lista.
- **insert(posicion, valor)**: inserta en *posicion* el valor indicado, desplazando los siguientes.
- **remove(elemento)**: elimina " **elemento** " de la lista. Para esto mismo podemos usar el "arcaísmo" **del**:
`del lista[4]`
- **copy()**: devuelve una copia de la lista. Podemos usarlo para crear una nueva si no queremos alterar la original.
- **reverse(lista)**: invierte el orden de la lista.
- **pop()**: Devuelve el último elemento de la lista y lo retira de la lista. También podemos usarlo para eliminar un elemento indicando el índice
`lista.pop(2)`

Esta función nos muestra que podemos usar una lista como soporte para un funcionamiento más complejo como sería una cola o pila, donde vamos eliminando el último elemento.

- **extend(lista2)**: añade a la lista actual el contenido de lista2. Es equivalente a hacer una suma de las dos listas:

```
python lista1.extend(lista2) lista1 = lista1 + lista2
```

* **sort()**: Ordena la lista. Si tenemos una lista que mezcla distintos tipos, por ejemplo números y letras nos dará una excepción de tipo **TypeError**.

También podemos ordenar una lista con la función **sorted**:

```
python  
sorted(list)
```

Recorrer una lista: enumerar

Uno de los usos más frecuentes de las listas es recorrer sus elementos. Ya dijimos que los bucles *for* están pensado para recorrer colecciones. Veamos cómo hacerlo con una lista:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for amigo in amigos:
    print(f'Hola {amigo}')
```

A este proceso de recorrer una lista se le denomina enumerar o iterar. A veces necesitamos además de iterar el indicar la posición del elemento. Podemos hacerlo con la palabra reservada **enumerate**, obteniendo tanto el elemento como su posición:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for posicion, amigo in enumerate(amigos): # Crea una variable "posicion" y
    otra "amigo" donde va poniendo el índice y el valor de los elementos
    print(posicion, amigo)
```

Este ejemplo nos muestra que podemos utilizar varias variables en un mismo bucle *for*, algo que haremos con frecuencia.

Creación dinámica de listas

En ocasiones necesitamos generar una lista a partir de otra, realizando una operación sobre sus elementos. Podemos hacerlo con lo que ya sabemos, recorriendo nuestra lista y añadiendo los elementos a una nueva lista:

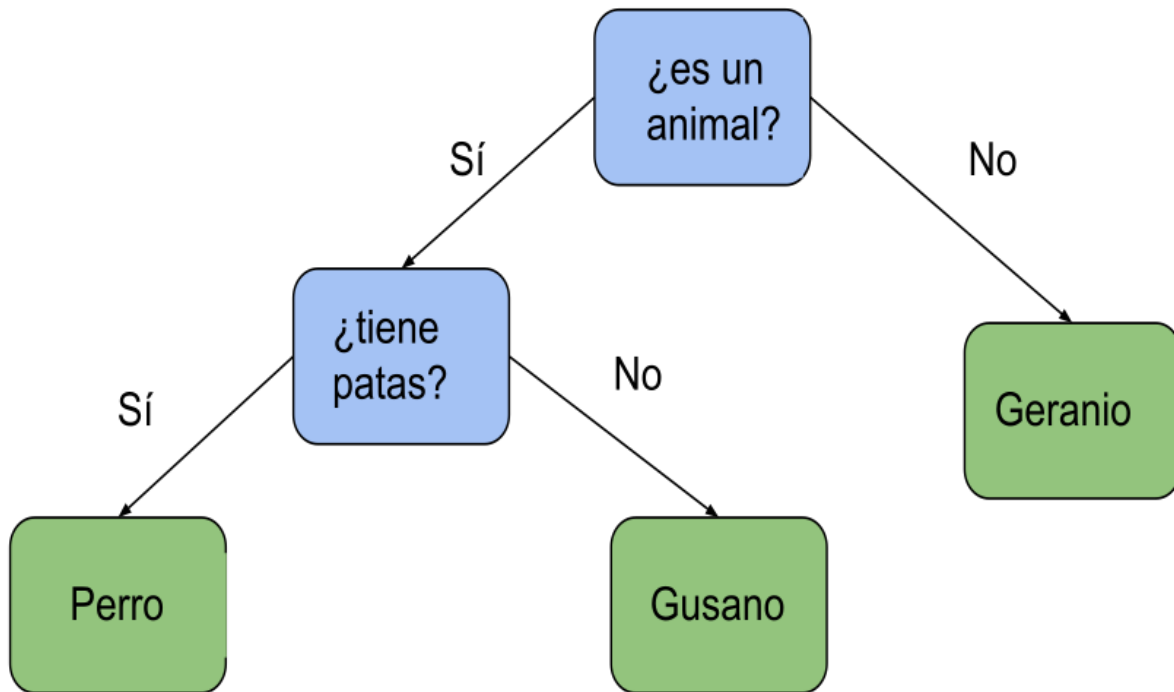
```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencia = [] # Creamos la lista vacía
for radio in radios:
    circunferencia.append(radio*pi)
```

Pero como ésto es algo muy frecuente, Python incorpora una manera muy cómoda de hacerlo, donde especificamos dentro de la lista la manera en la que calculamos cada elemento a partir de un bucle *for*:

```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencias = [pi * num for num in radios]
```

Ejemplo: Juego de las 20 preguntas

Vamos a usar ahora una lista para almacenar una estructura compleja como es un árbol:



Este árbol nos va a permitir navegar entre los nodos que contendrán tanto las preguntas como las respuestas. Cada nodo también guarda referencias a la posición de otros dos:

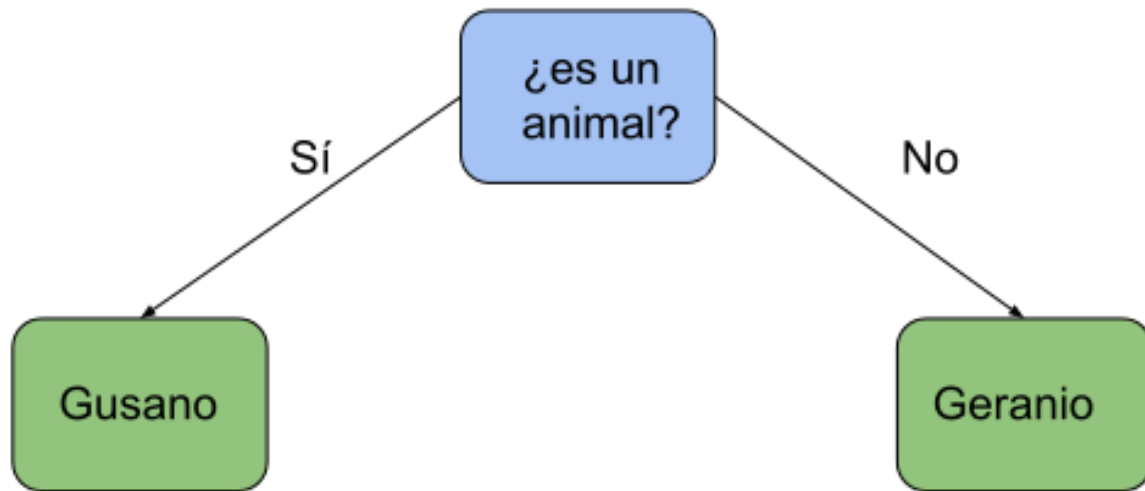
- Al nodo al que ir si la respuesta es *Sí*.
- Nodo al que ir si la respuesta es *No*.

Los nodos que son preguntas tendrán valores válidos para moverse, los llamaremos **nodos rama** y actúan como bifurcación. Los nodos respuesta tienen el valor -1 en los nodos a los que moverse, diremos que estos nodos son **terminales** o **nodo hoja**.

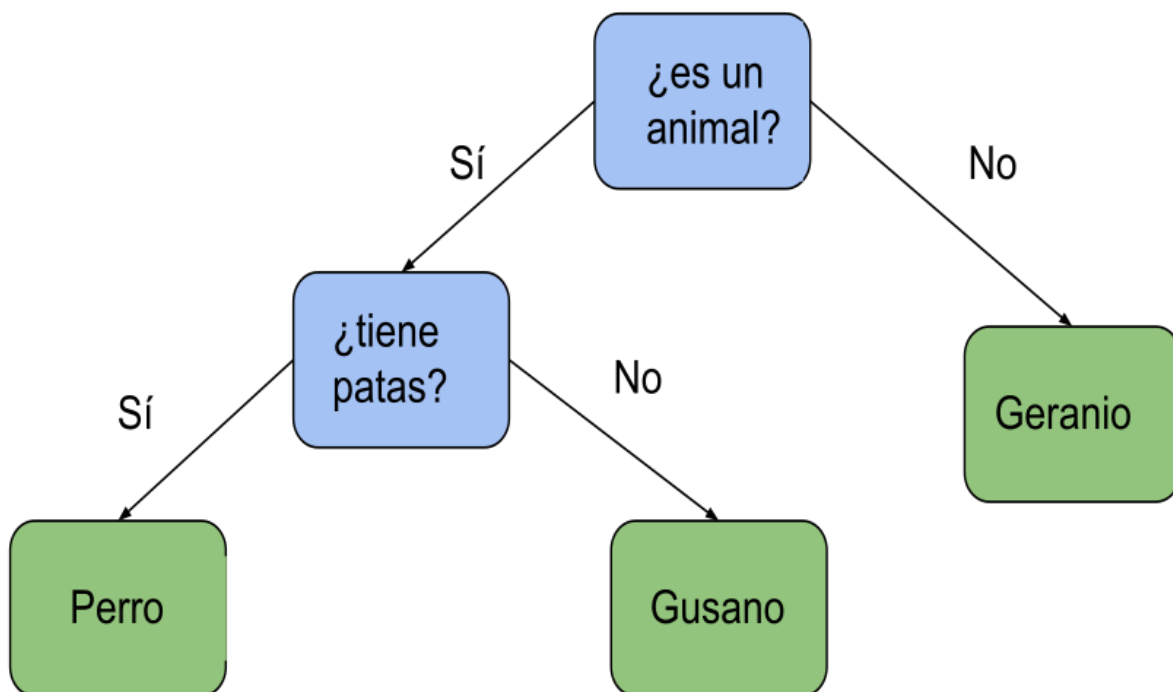
Iremos moviéndonos entre nodos hasta que lleguemos a un nodo terminal que será una respuesta y preguntaremos si es correcta.

Si no lo es, cambiaremos el nodo actual que pasa de ser terminal a ser bifurcación y añadiremos 2 nodos terminales que serán uno la respuesta actual y otro la nueva respuesta. El nodo actual ahora contendrá una nueva pregunta que apuntará a las dos posibles respuestas.

Este es el ejemplo de cambio cuando preguntamos si Gusano es la respuesta y no es el caso.



Este será el árbol después de añadir la nueva pregunta:



Este será el contenido de nuestra lista, donde vemos que los nodos *Respuesta* no conducen a ningún otro y por eso tienen los valores -1, mientras que los nodos *Pregunta* apuntan a otros dos.

id	text	Sí	No
0	es un animal	2	1
1	geranio	-1	-1

2	tiene patas	4	3
3	gusano	-1	-1
4	perro	-1	-1

Este programa lo vamos a ir ampliando a lo largo del curso, añadiendo funcionalidad y mejorando la forma en la que almacenamos la información.

Ejemplo: Juego de las 20 preguntas v1

Matrices

Hemos dicho que una lista puede contener cualquier cosa. De hecho podemos hacer una lista que contenga listas, es lo que llamamos una matriz. Veamos un ejemplo

```
fila1 = [1, 3, 5]
fila2 = [9, 3, 5]
fila3 = [7, 3, 8]
```

```
matriz = [fila1, fila2, fila3]
```

```
print(matriz) # imprime la matriz
```

completa

```
print(matriz[1][2]) # imprime un solo elemento el 3º de la fila 2º
```

Como vemos necesitamos 2 índices para acceder a un elemento dado.

Diccionarios

Un diccionario es una colección de pares etiqueta/valor, separados por el signo ":". También se usa la palabra clave o llave para referirse a la etiqueta.

Se usan las llaves, expresados de la siguiente forma (usaremos el que parece ser el ejemplo por excelencia de los diccionarios: los lenguajes de programación y su fecha de creación):

```
>>> lenguajes = { 'C':1972, 'python':1991, 'Java':1996}
>>> print(lenguajes)
{'C': 1972, 'python': 1991, 'Java': 1996}
```

Podemos crear un diccionario vacío con:

```
diccionario = {}
```

Para recuperar un valor preguntaremos por una llave:

```
>>>
print(lenguajes['C'])
1972
```

Si el elemento no existe se produce una excepción de tipo **KeyError**.

Podemos evitar el error usando `get('clave')` que devolverá **None** si la clave no está en el diccionario. **None** es una palabra reservada que indica cuando una variable no tiene ningún valor.

Podemos pasarle al método `get` un argumento que se devolverá como valor por defecto si la clave no existe:

```
>>> print(lenguajes.get('C++', 'Año desconocido')) 'Año desconocido'
```

Si queremos obtener un elemento y a la vez retirarlo del diccionario usaremos la función **pop('clave')**. Por ejemplo:

```
>>> lenguajes = { 'C':1972, 'python':1991, 'Java':1996}
>>> lenguajes.pop('Java')
1996
>>> print(lenguajes)
{'C': 1972, 'python': 1991}
```

Otra forma de ver si una clave está en el diccionario es usando **in**:

```
>>> 'BASIC' in
lenguajes False
```

Asignando un nuevo elemento se incluirá en el diccionario:

```
>>> lenguajes['C++'] = 1983
```

Podemos borrar un elemento usando la función **del**:

```
>>> del lenguajes['Java']
```

Podemos crear diccionarios con esta otra sintaxis, donde usamos la palabra reservada **dict()** y si queremos, los elementos que va a contener, indicado **clave=valor**:

```
>>> lenguajes = dict(C=1972, Java=1996, Python=1991)
>>> print(lenguajes)
{'C': 1972, 'Java': 1996, 'Python': 1991}
```

O a partir de listas, asignando a todos el mismo valor:

```
>>> lenguajesAntiguos = ['Assembler','Lisp','Pascal']
lenguajesAntiguos = dict.fromkeys(oldLenguajes,0)
>>> print(lenguajesAntiguos)
{'Assembler': 0, 'Lisp': 0, 'Pascal': 0}
```

Iterando en los diccionarios

Podemos iterar un diccionario usando las claves:

```
>>> for clave in lenguajes:
...     print(clave)
C
Java
a
Pyt
hon
```

También podemos iterar por los valores usando la función *values()* (Cuidado con olvidar los paréntesis de *values()*, que es una función):

```
>>> for valor in lenguajes.values():
...     print(valor)
1972
1996
1991
```

Del mismo modo que hemos accedido a todos los valores almacenados en el diccionario, también podemos acceder a todas las claves usando la función *keys*

```
>>> for key in lenguajes.keys():
...     print(key)
C
Java
a
Pyt
hon
>>> 'C++' in
lenguajes.keys() False
```

Aunque no es una lista, podemos usarla como tal.

Incluso podemos iterar por ambos a la vez usando la función *items()* (Cuidado con olvidar los paréntesis de *items()*, que es una función):

```
>>> for clave,valor in lenguajes.items():
...     print(clave,' - > ',valor)
C - > 1972
python - > 1991
Java - > 1996
```

Ordenación en los diccionarios

Si intentamos ordenar nuestro diccionario

```
>>> lenguajes = {'Pascal':1970, 'Java':1996, 'Python':1991, 'C':1972}
>>> sorted(lenguajes)
```

```
['C', 'Java', 'Pascal', 'Python']
```

Vemos que nos lo ordena las llaves (keys)

Si lo que queremos es que nos ordene por los valores podemos usar el argumento "key" de la función sorted, indicándole la función que queremos que use, en este caso la función "get" que nos da el valor para una llave

```
>>> sorted(lenguajes, key = lenguajes.get)
['Pascal', 'C', 'Python', 'Java']
```

Cadenas y colecciones

Después de trabajar con listas podemos ver las cadenas como una lista de caracteres:

- Accedemos a cada carácter por índice `cndcnn[4]`.
- Podemos seleccionar una parte de una cadena `cndcnn[inicio:fin]`.

Veamos algunas funciones para trabajar con cadenas.

split: rompe una cadena en palabras

```
>>> frase = 'hola mundo'
>>> palabras = frase.split()
>>> print(palabras)
['hola', 'mundo']
```

Se le puede pasar como argumento el carácter separador, que por defecto es espacio.

join: genera cadena

Genera una cadena a partir de un separador y una lista

```
>>> " ".join(['hola', 'mundo']) # genera una cadena de texto donde se concatenan
los elementos de la lista separados por la cadena, un espacio en este caso
'hola mundo'
>>> personas = ['Juan', 'Pepe', 'Ana', 'Luisa']
>>> ', '.join(personas)
'Juan, Pepe, Ana, Luisa'
```

Buscar con *find*

El método *find* nos permite buscar dónde aparece una cadena dentro de otra. Nos devuelve la posición dentro de la cadena en la que aparece la subcadena. En caso no de aparecer devuelve -1:

```
>>> cadena = 'hola, caracola'
>>> cadena.find('ola')
1
>>> cadena.find('bola')
-1
```

También podemos hacer que se busque empezando por el final con *rfind*:

```
>>> cadena = 'hola, caracola'
>>> cadena.rfind('ola')
11
```

Ejemplos de cadenas y colecciones

Conteo de caracteres de un texto

Creamos una función que guarda en un diccionario la cantidad de veces que aparece un carácter dado en un texto

```
def countChars(texto):
    """
    Determina el número de ocurrencias de los caracteres de "texto", no
    cuenta los caracteres salto de línea "\n"
    :param texto: texto a analizar
    :returns: devuelve un diccionario con el número de ocurrencias de cada
    carácter
    """
    freq = dict() # crea el diccionario freq
    for c in texto: # itera sobre todos los caracteres de la cadena "texto"
        y guarda en 'c' un carácter cada vez
        if c == '\n': # Nos saltamos los caracteres retornos de carro '\n'
            continue # saltamos al inicio del bucle
        if c in freq: # si ya está
            freq[c] += 1 # lo incrementamos
```

```

        else: # es la primera aparición
            freq[c] = 1 # añadimos con 1 ocurrencia
    # print(freq) # imprime el diccionario
    return freq # devuelve el diccionario freq y termina la función

```

Cálculo de frecuencia de cada carácter en un texto

Usamos la función anterior para contar los caracteres y a partir del diccionario calculamos la frecuencia:

```

def freqChars(texto):
    """
    Contamos los caracteres de un texto,
    calculamos la frecuencia y comprobamos que están normalizadas
    normalizamos y comprobamos la normalización
    :param texto: texto a analizar
    :returns: devuelve un diccionario con la frecuencia de ocurrencias de
    cada carácter
    """

    numeroCaracteres = len(texto)
    suma = 0

    freq = countChars(texto)

    for key in freq:
        freq[key] *= 1.0/numeroCaracteres
        suma += freq[key]
    errorSuma = abs(suma-1)
    if errorSuma > 2:
        print("Error de normalización suma = ",str(suma))

    return freq

```

Traducción de caracteres

Cuando trabajamos con algoritmos de encriptación sencillos podemos usar un diccionario para traducir el texto original en el texto codificado:

```

pt = dict()
pt['a'] = '7'
pt['e'] = ';'
pt['i'] = ' '
pt['o'] = '.'
pt['u'] = 'j'
# ....

```

A partir de un texto, generamos el texto codificado usando el diccionario que contiene la clave.

Conjuntos/Sets

También existen los **Conjuntos** (sets), que almacenan valores sin ordenar y que no pueden estar repetidos. Sólo se pueden guardar elementos inmutables (que no cambian) y de los que se puede obtener un valor **hash** (algo similar a un id único).

La diferencia entre conjunto y lista es que en los conjuntos **no se repiten elementos**.

Además en los conjuntos no podemos acceder a los elementos individualmente.

Como su nombre indica, están pensados para trabajar las típicas operaciones entre conjuntos: uniones, intersecciones, diferencias,...

Los crearemos con los elementos entre llaves:

```
>>> s = {1, 2, 4}
```

También podemos crearlos a partir de otra colección usando la función `set(coleccion)`:

```
listaNombres = 'Ana Pepe Juan Ana'.split() # se crea una lista de 4 elementos
print(listaNombres)
conjuntoNombres = set(listaNombres)
print(conjuntoNombres)
```

Evidentemente al convertirlo se eliminan los elementos repetidos, como podemos ver en el ejemplo anterior.

También podemos añadir elementos con el método **add**:

```
>>> print(s)
>>> s.add(5)
>>> print(s)
>>> s.add(5)
>>> print(s)
{1, 2, 4}
{1, 2, 4, 5}
{1, 2, 4, 5}
```

Y vemos que si intentamos añadir un elemento repetido, sólo aparecerá una vez.

Operaciones lógicas con conjuntos

La principal utilidad de los conjuntos es hacer operaciones lógicas entre ellos, como por ejemplo esta operación **OR** utilizando el símbolo `|` en la que se hace una **unión** o **suma** de elementos:

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5, 6}
>>> a | b
{1, 2, 3, 4, 5, 6}
```

Sería equivalente a una *unión*, en la que se hace un conjunto que contiene a todos los elementos de ambos. Por ello podemos hacerlo también con el método *union()*:

```
>>> a.union(b)
{1, 2, 3, 4, 5, 6}
```

No importa el orden en el que hagamos la operación *unión*, "*n | b*" será idéntica a "*b | n*".

También podemos hacer una **diferencia**, que nos mostrará aquellos que están en el primero y no el segundo, es decir, eliminamos del primero los elementos del segundo:

```
>>> a-b
{1, 2}
```

En el caso de la *diferencia*, sí que importa el orden en el que hacemos la operación y vemos que el resultado es distinto

```
>>> b-a
{5, 6}
```

También podemos hacer una operación **AND** que nos dará los elementos que se repiten en ambos conjuntos, para lo que usaremos el operador "&":

```
>>> b&a
{3, 4}
```

La operación **XOR** nos devuelve los elementos que no se repiten, usando el operador "^":

```
>>> a^b
{1, 2, 5, 6}
```

Utilidades - Zip

Al contrario de lo que pueda pareceros el nombre, la función **zip** no sirve para comprimir cosas sino a encajarlas. Quizás porque *zip* significa cremallera en inglés.

A partir de dos listas de elementos podemos generar una lista formada por pares de elementos, uno de cada lista

```
países = ['Francia', 'Alemania', 'España']
capitales = ['París', 'Berlín', 'Madrid']
```

```
for país, capital in zip(países, capitales):
    print(f'{país} capital {capital}')
```

También podemos generar un diccionario

```
diccionario = dict(zip(países, capitales))
print(diccionario)
```

Ejercicios colecciones

Vamos a ver algunos ejercicios para practicar lo aprendido con las colecciones.

Uniando 2 listas en un diccionario

Creamos un diccionario a partir de 2 listas: una contendrá las claves y otra los valores:

```
keys = ['Ten', 'Twenty', 'Thirty'] # Genera la lista keys
values = [10, 20, 30] # Genera la lista values

diccionario = {} # Genera un diccionario vacío.
contador = 0 # lo usaremos como índice para recorrer los valores
for key in keys: # itera la lista keys guardando en key un valor cada vez
    diccionario[key] = values[contador] # añade al diccionario el valor al
    que apunta "values[contador]" con clave "key"
    contador += 1 # incrementa el índice

print(diccionario)
```

Mezclar 2 diccionarios en uno

Unimos 2 diccionarios en uno que contendrá todos los pares:

```
dict1 = {'Ten': 10, 'Twenty': 20, 'Thirty': 30}
dict2 = {'Thirty': 30, 'Forty': 40, 'Fifty': 50}

dict3 = dict1.copy() # Creamos un nuevo diccionario
dict3.update(dict2)
print(dict3)
```

Acceso a un elemento dentro de un diccionario

Se pueden crear estructuras anidadas dentro de un diccionario: el valor es un nuevo diccionario.

Acceder al valor del elemento 'history' en un diccionario que tiene esta estructura.

```
sampleDict = {  
    "class":{  
        "student":{  
            "name":"Mike",  
            "marks":{  
                "physics":70,  
                "history":80  
            }  
        }  
    }  
}
```

```
sampleDict["class"]['student']['marks']['history']
```

Eliminar determinados pares de un diccionario

Vamos a ver cómo eliminar varios elementos de un diccionario:

```
diccionario = {  
    "name": "Kelly",  
    "age":25,  
    "salary": 8000,  
    "city": "New york"  
}  
  
clavesAQuitar = ["name", "salary"]  
  
restantes = {}  
  
for key in diccionario:  
    if key not in clavesAQuitar:  
        restantes[key] = diccionario[key]  
  
print(restantes)
```

Ejemplo: Jugando con barajas

En este ejemplo vamos a utilizar lo aprendido sobre manejo de cadenas y su tratamiento como distintas colecciones.

Vamos a crear una estructura de funciones y variables para trabajar con

naipes. Código: Juego de cartas

Indicación de Tipos a partir de Python 3.5

A partir de la versión 3.5 de Python, podemos indicar el tipo que va a tener una variable. Sigue sin ser necesario hacerlo pero aporta ventajas como el evitar errores en nuestro código.

Podemos indicar el tipo que tendrá un argumento o comprobar si se cumple que cierta variable sea de ese tipo.

```
diccionario = Dict[str,str]
direccion = Tuple[str,int]
agenda = List[str]
```

También se pueden definir alias para tipos como por ejemplo

```
vector = list[float]
```

Ejecutando desde la consola

Ya vamos teniendo programas cada vez mayores y va siendo natural poder ejecutarlos por sí mismos, desde la consola, sin usar el IDE de desarrollo.

Ya hablamos en los temas iniciales sobre la instalación de la consola que nos va a permitir ejecutarlos.

Una vez instalada, podemos ejecutar nuestros ficheros de código simplemente escribiendo en la consola:

```
python3 fichero.py
```

Uso de funciones lambda con colecciones

Ya vimos los conceptos básicos, sintaxis, ejemplos simples y limitaciones de las lambda, enfocándonos en operaciones directas. Ahora, vamos a ver cómo las funciones lambda pueden integrarse con colecciones comunes de Python, como listas, tuplas, diccionarios y conjuntos.

Recordemos que las colecciones son estructuras de datos que almacenan múltiples elementos, como listas (ordenadas y mutables), tuplas (ordenadas e inmutables), diccionarios (basados en claves-valor) y conjuntos (no ordenados y sin duplicados). Las lambda son particularmente útiles aquí para proporcionar lógica personalizada en operaciones como ordenamiento, búsqueda de extremos o transformación condicional, todo en una forma concisa.

Introducción al Uso de Lambda con Colecciones

Las funciones lambda brillan cuando se combinan con colecciones porque permiten definir comportamientos "sobre la marcha" sin necesidad de funciones completas. Por ejemplo, puedes usar una lambda para especificar cómo ordenar elementos basándote en un criterio personalizado, o para encontrar el máximo/mínimo en una colección según una condición.

Ventajas en este contexto:

- **Personalización rápida:** Define claves de ordenamiento o comparación sin código extra.
- **Eficiencia:** Ideal para operaciones de un solo uso en datos almacenados en colecciones.
- **Legibilidad:** Mantiene el código compacto cuando la lógica es simple.

Empezaremos con listas, ya que son las más flexibles, y luego pasaremos a otras colecciones.

Lambda con Listas

Las listas son colecciones ordenadas y mutables. Las lambda se usan comúnmente para modificar el comportamiento de métodos o funciones que operan sobre ellas, como ordenar o encontrar valores extremos.

Ejemplo fi: Ordenar una lista con sorted() usando lambda como clave

La función `sorted()` devuelve una nueva lista ordenada. Puedes pasar una lambda como el argumento `key` para definir el criterio de ordenamiento.

```
# Lista de números
numeros = [5, 2, 8, 1, 9]

# Ordenar ascendentemente (por defecto, pero usando lambda para ilustrar)
ordenada = sorted(numeros, key=lambda x: x)
print(ordenada) # Salida: [1, 2, 5, 8, 9]

# Ordenar descendientemente invirtiendo el valor
ordenada_desc = sorted(numeros, key=lambda x: -x)
print(ordenada_desc) # Salida: [9, 8, 5, 2, 1]
```

Aquí, la lambda `lambda x: -x` transforma cada elemento negándolo, lo que invierte el orden natural.

Otro ejemplo con una lista de cadenas: Ordenar por longitud.

```
palabras = ["manzana", "pera", "kiwi", "naranja"]
ordenada_por_longitud = sorted(palabras, key=lambda s: len(s))
print(ordenada_por_longitud) # Salida: ['pera', 'kiwi', 'manzana', 'naranja']
```

La lambda `lambda s: len(s)` usa la longitud de cada cadena como clave de orden.

Ejemplo 2: Ordenar in-place con `list.sort()`

El método `sort()` modifica la lista original. Similar a `sorted()`, acepta una lambda como `key`.

```
frutas = ["banana", "uva", "fresa", "mango"]
frutas.sort(key=lambda f: f.lower()) # Ordenar ignorando mayúsculas
print(frutas) # Salida: ['banana', 'fresa', 'mango', 'uva']
```

Ejemplo 3: Encontrar máximo/mínimo con `max()` y `min()` usando lambda

Estas funciones pueden usar una lambda para definir la clave de comparación.

```
puntos = [10, 25, 5, 30]
maximo = max(puntos, key=lambda p: p)
print(maximo) # Salida: 30 (equivalente al máximo natural)

# Para un caso más interesante: Máximo basado en módulo
max_modulo = max(puntos, key=lambda p: p % 20)
print(max_modulo) # Salida: 10 (10 % 20 = 10, el mayor en este criterio)
```

De manera similar para `min()`.

Lambda con Tuplas

Como ya hemos visto, las tuplas son inmutables, pero se pueden ordenar o analizar de forma similar a las listas. A menudo, las funciones lambda se usan para acceder a elementos específicos en tuplas anidadas.

Ejemplo: Ordenar una lista de tuplas

Supongamos una lista de tuplas representando (nombre, edad).

```
personas = [("Ana", 28), ("Bob", 22), ("Carlos", 35)]
ordenada_por_edad = sorted(personas, key=lambda p: p[1])
print(ordenada_por_edad) # Salida: [('Bob', 22), ('Ana', 28), ('Carlos', 35)]
```

La lambda `lambda p: p[1]` accede al segundo elemento (edad) de cada tupla.

Otro ejemplo: Ordenar por nombre (primer elemento).

```
ordenada_por_nombre = sorted(personas, key=lambda p: p[0].lower())
print(ordenada_por_nombre) # Salida: [('Ana', 28), ('Bob', 22), ('Carlos', 35)]
```

Ejemplo: Máximo en tuplas anidadas

```
coordenadas = [(1, 5), (3, 2), (4, 7)]
max_y = max(coordenadas, key=lambda c: c[1])
print(max_y) # Salida: (4, 7) (mayor valor en y)
```

4. Lambda con Diccionarios

Los diccionarios almacenan pares clave-valor. Las funciones lambda ayudan a ordenar por claves o valores, o a procesar items.

Ejemplo: Ordenar items de un diccionario

Usa `sorted()` sobre `dict.items()` para obtener una lista de tuplas ordenadas.

```
precios = {"manzana": 1.5, "banana": 0.8, "naranja": 1.2}
ordenado_por_precio = sorted(precios.items(), key=lambda item: item[1])
print(ordenado_por_precio) # Salida: [('banana', 0.8), ('naranja', 1.2), ('manzana', 1.5)]
```

La lambda `lambda item: item[1]` selecciona el valor para ordenar.

Ejemplo: Encontrar clave con valor máximo

```
calificaciones = {"Alice": 85, "Bob": 92, "Charlie": 78}
mejor = max(calificaciones.items(), key=lambda est: est[1])
print(mejor) # Salida: ('Bob', 92)
```

Aquí, `max()` usa la lambda para comparar por el valor (calificación).

5. Lambda con Conjuntos

Los conjuntos son no ordenados y sin duplicados, por lo que no se pueden ordenar directamente. Sin embargo, puedes convertirlos a listas y aplicar lambda.

Ejemplo: Ordenar un conjunto convertido a lista

```
numeros_set = {3, 1, 4, 1, 5, 9} # Se elimina duplicado automáticamente
ordenados = sorted(list(numeros_set), key=lambda n: n)
print(ordenados) # Salida: [1, 3, 4, 5, 9]
```

O para un criterio personalizado, como ordenar por paridad (pares primero).

```
ordenados_paridad = sorted(list(numeros_set), key=lambda n: n % 2)
print(ordenados_paridad) # Salida: [4, 1, 3, 5, 9] (pares %2=0 primero)
```

Ejemplo: Mínimo en un conjunto

```
minimo = min(numeros_set, key=lambda n: n * -1) # Invierte para encontrar
"máximo" como mínimo
print(minimo) # Salida: 9 (ya que -9 es el menor después de multiplicar por
-1)
```

6. Ejemplos Avanzados: Combinaciones y Closures con Colecciones

Ejemplo: Lambda con condicional en ordenamiento

Ordenar una lista de números priorizando positivos.

```
mixtos = [-3, 5, -1, 2, -4]
ordenados_positivos_primeros = sorted(mixtos, key=lambda x: (x < 0, abs(x)))
print(ordenados_positivos_primeros) # Salida: [2, 5, -1, -3, -4] (positivos
primero, luego negativos por absoluto)
```

La lambda devuelve una tupla: (True/False para negativo, valor absoluto), lo que agrupa por signo y luego ordena.

7. Buenas prácticas con colecciones

- **Elige la colección adecuada:** Usa listas para mutabilidad, tuplas para inmutabilidad, diccionarios para búsquedas rápidas y conjuntos para unicidad.
- **Evita complejidad en funciones lambda:** Si la clave requiere mucha lógica, define una función normal.
- **Considera rendimiento:** Para colecciones grandes, lambda no afecta mucho, pero asegúrate de que la expresión sea eficiente.
- **Prueba con datos reales:** Experimenta con tus propias colecciones para ver resultados.

Conclusión

En esta segunda parte, hemos visto cómo las funciones lambda elevan el manejo de colecciones en Python, permitiendo operaciones personalizadas como ordenamiento y búsqueda de extremos de manera concisa. Combinadas con listas, tuplas, diccionarios y conjuntos, las lambda hacen que tu código sea más expresivo y eficiente.

Comprensión de listas o Comprehension Lists

El concepto de **compresión de listas** (o *list comprehension* en inglés) es una característica poderosa y concisa de algunos lenguajes de programación, como Python, que permite crear listas de manera eficiente y legible en una sola línea de código. En lugar de usar bucles tradicionales para construir una lista elemento por elemento, la compresión de listas proporciona una sintaxis compacta para generar listas basadas en iterables existentes, aplicando transformaciones o filtros según sea necesario.

A continuación, explico en detalle el concepto, su estructura, usos, ventajas y ejemplos prácticos, centrándome principalmente en Python, ya que es el lenguaje donde este concepto es más conocido y utilizado.

¿Qué es una compresión de listas?

Una compresión de listas es una construcción sintáctica que permite crear una nueva lista a partir de un iterable (como una lista, tupla, rango, etc.) aplicando una expresión a cada elemento del iterable, opcionalmente filtrando los elementos que cumplen ciertas condiciones. En Python, se escribe dentro de corchetes `[ ]` y combina una expresión con un bucle `for` (y, en algunos casos, una condición `if`).

La compresión de listas es una forma de expresar de manera declarativa lo que se podría lograr con un bucle `for` tradicional, pero en una sola línea, lo que mejora la legibilidad y reduce el código.

Sintaxis básica

La sintaxis general de una compresión de listas en Python es:

```
[expresión for elemento in iterable if condición]
```

- **expresión:** Es la operación o transformación que se aplica a cada elemento del

iterable. Por ejemplo, `elemento * 2` o `elemento.upper()`.

- **elemento**: Es la variable que toma cada valor del iterable en cada iteración.
- **iterable**: Es la colección o secuencia de datos sobre la que se itera (por ejemplo, una lista, un rango, una cadena, etc.).
- **if condición** (opcional): Filtra los elementos del iterable, incluyendo sólo aquellos que cumplen la condición.

Ejemplo simple:

Supongamos que queremos crear una lista con los cuadrados de los números del 1 al 5.

Con un bucle tradicional:

```
cuadrados = []
for num in range(1, 6):
    cuadrados.append(num ** 2)
print(cuadrados) # Salida: [1, 4, 9, 16, 25]
```

Con compresión de listas:

```
cuadrados = [num ** 2 for num in range(1, 6)]
print(cuadrados) # Salida: [1, 4, 9, 16, 25]
```

La compresión de listas logra el mismo resultado en una sola línea, siendo más concisa.

Componentes avanzados

Las compresiones de listas pueden ser más complejas, incluyendo múltiples bucles for y condiciones if. A continuación, se describen los casos más comunes:

a) Compresión con una condición: Puedes filtrar elementos usando una condición if. Por ejemplo, si quieres solo los cuadrados de los números pares del 1 al 10:

```
cuadrados_pares = [num ** 2 for num in range(1, 11) if num % 2 == 0]
print(cuadrados_pares) # Salida: [4, 16, 36, 64, 100]
```

b) Múltiples bucles for: Puedes anidar bucles dentro de una compresión de listas para generar combinaciones. Por ejemplo, para crear una lista con todas las combinaciones de dos listas:

```
letras = ['a', 'b']
numeros = [1, 2]
combinaciones = [letra + str(numero) for letra in letras for numero in numeros]
print(combinaciones) # Salida: ['a1', 'a2', 'b1', 'b2']
```

Esto es equivalente al siguiente bucle anidado:

```
combinaciones = []
for letra in letras:
    for numero in numeros:
        combinaciones.append(letra + str(numero))
```

c) Condiciones múltiples: Puedes combinar varias condiciones usando operadores lógicos. Por ejemplo, obtener números que sean pares y mayores a 5:

```
numeros_filtrados = [num for num in range(1, 11) if num % 2 == 0 and num > 5]
print(numeros_filtrados) # Salida: [6, 8, 10]
```

d) Expresiones condicionales (if-else): Puedes usar una expresión condicional para transformar elementos de manera diferente según una condición. Por ejemplo, clasificar números como "par" o "impar":

```
clasificacion = ["par" if num % 2 == 0 else "impar" for num in range(1, 6)]
print(clasificacion) # Salida: ['impar', 'par', 'impar', 'par', 'impar']
```

Ventajas de la compresión de listas

- **Concisión:** Reduce la cantidad de código necesario en comparación con los bucles tradicionales.
- **Legibilidad:** Cuando se usa correctamente, hace que el código sea más claro y fácil de entender.
- **Eficiencia:** En Python, las compresiones de listas suelen ser más rápidas que los bucles tradicionales porque están optimizadas a nivel interno.
- **Funcionalidad expresiva:** Permite expresar transformaciones y filtros complejos en una sola línea.

Limitaciones y buenas prácticas

Aunque las compresiones de listas son muy útiles, no siempre son la mejor opción. Aquí algunas consideraciones:

- **Legibilidad en casos complejos:** Si la compresión se vuelve demasiado complicada (por ejemplo, con muchos bucles anidados o condiciones), puede ser más difícil de leer que un bucle tradicional. En estos casos, es mejor usar un bucle explícito.

- **No mutan listas existentes:** Las compresiones de listas siempre crean una nueva lista; no modifican listas existentes.
- **Limitado a listas:** Aunque Python también tiene compresiones para diccionarios, conjuntos y generadores, la compresión de listas está restringida a crear listas.

Buenas prácticas: Usa compresiones de listas para operaciones simples y claras. Si necesitas depuración o lógica compleja, opta por un bucle tradicional.

Ejemplos prácticos

Aquí hay algunos ejemplos adicionales para ilustrar la versatilidad de las compresiones de listas:

a) Extraer caracteres de una cadena

```
texto = "Hola Mundo"
letras = [char for char in texto if char.isalpha()]
print(letras) # Salida: ['H', 'o', 'l', 'a', 'M', 'u', 'n', 'd', 'o']
```

b) Crear una matriz (lista de listas)

```
matriz = [[i * j for j in range(1, 4)] for i in range(1, 4)]
print(matriz) # Salida: [[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

c) Filtrar y transformar al mismo tiempo

```
palabras = ["sol", "luna", "estrella", "cielo"]
largas_mayus = [pal.upper() for pal in palabras if len(pal) > 3]
print(largas_mayus) # Salida: ['ESTRELLA', 'CIELO']
```

Relación con otras compresiones en Python

Además de las compresiones de listas, Python ofrece otras construcciones similares: -

Compresión de diccionarios: {clave: valor for elemento in iterable} -

Compresión de conjuntos: {elemento for elemento in iterable} - **Expresiones**

generadoras: (expresión for elemento in iterable)

Por ejemplo, una compresión de diccionario equivalente al ejemplo de cuadrados:

```
cuadrados_dict = {num: num ** 2 for num in range(1, 6)}
print(cuadrados_dict) # Salida: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Conclusión

La compresión de listas es una herramienta poderosa en Python que permite crear listas de manera concisa y expresiva. Es ideal para transformaciones y filtrados simples, pero debe usarse con moderación para evitar sacrificar la legibilidad en casos complejos. Su sintaxis compacta, combinada con su eficiencia y claridad, la convierte en una de las características más apreciadas del lenguaje.